Deep Reinforcement Learning

Unit 1: Foundations and First Agent

Hugging Face Deep RL Course

Faithful MDX-to-LaTeX study edition

Original content: Hugging Face Deep RL Course.

Contents

1 Introduction to Deep Reinforcement Learning **2**

2 What is Reinforcement Learning? **3**

2.1 The big picture . . . 3

3 The Reinforcement Learning Framework **5**

3.1 The RL Process . . . 5

3.2 The reward hypothesis: the central idea of Reinforcement Learning . . . 6

3.3 Markov Property . . . 6

3.4 Observations/States Space . . . 6

3.5 Action Space . . . 8

3.6 Rewards and the discounting . . . 9

4 Type of tasks **11**

4.1 Episodic task . . . 11

4.2 Continuing tasks . . . 12

5 The Exploration/Exploitation trade-off **13**

6 Two main approaches for solving RL problems **15**

6.1 The Policy π: the agent's brain . . . 15

6.2 Policy-Based Methods . . . 15

6.3 Value-based methods . . . 18

7 The "Deep" in Reinforcement Learning **20**

8 Summary **21**

9 Glossary **21**

10 Train your first Deep Reinforcement Learning Agent **22**

11 Unit 1: Train your first Deep Reinforcement Learning Agent **24**
11.1 Objectives of this notebook . . . 24
11.2 This notebook is from Deep Reinforcement Learning Course . . . 25
11.3 Prerequisites . . . 25
11.4 A small recap of Deep Reinforcement Learning . . . 26

12 Let's train our first Deep Reinforcement Learning agent and upload it to the Hub **26**
12.1 Get a certificate . . . 27
12.2 Set the GPU . . . 27
12.3 Install dependencies and create a virtual screen . . . 27
12.4 Import the packages . . . 28
12.5 Understand Gymnasium and how it works . . . 29
12.6 Create the LunarLander environment and understand how it works . . . 30
12.7 Create the Model . . . 32
12.8 Train the PPO agent . . . 33
12.9 Evaluate the agent . . . 33
12.10Publish our trained model on the Hub . . . 34
12.11Load a saved LunarLander model from the Hub . . . 37
12.12Some additional challenges . . . 38
12.13Keep learning, stay awesome . . . 39

13 Quiz **39**
13.1 Answer Key and Explanations . . . 40

14 Conclusion **42**

15 Additional Readings **43**
15.1 Deep Reinforcement Learning . . . 43
15.2 Gym . . . 43

1 Introduction to Deep Reinforcement Learning

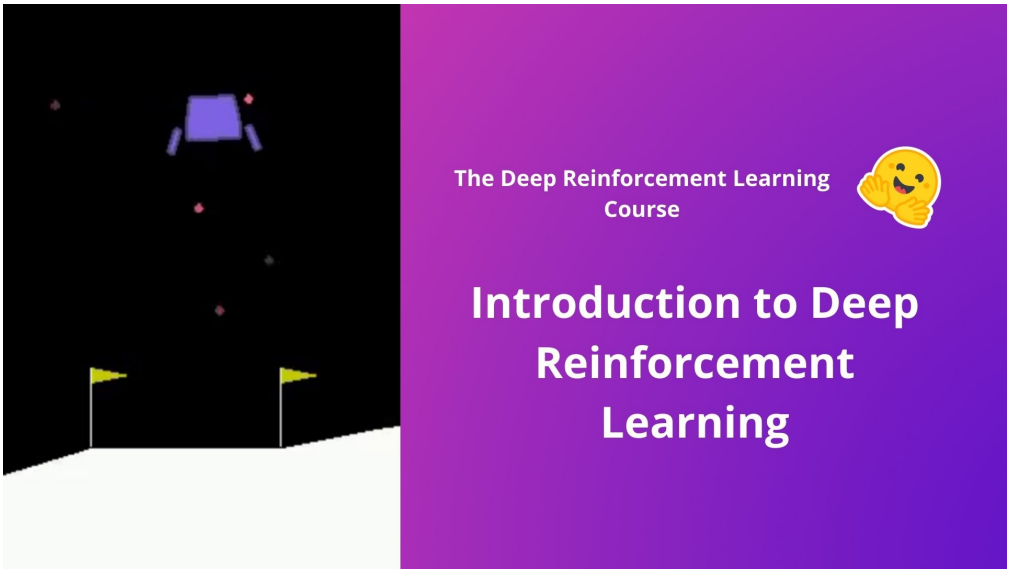


Figure 1: Unit 1: Introduction to Deep Reinforcement Learning (official course illustration)

Welcome to the most fascinating topic in Artificial Intelligence: **Deep Reinforcement Learning.**

Deep RL is a type of Machine Learning where an agent learns **how to behave** in an environment **by performing actions** and **seeing the results.**

In this first unit, **you'll learn the foundations of Deep Reinforcement Learning.**

Then, you'll **train your Deep Reinforcement Learning agent, a lunar lander to land correctly on the Moon** using Stable-Baselines3, a Deep Reinforcement Learning library.

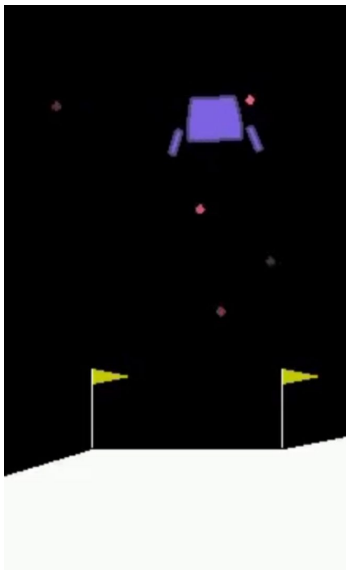


Figure 2: LunarLander environment: static scene from the official course thumbnail; the original animation is linked below (original GIF)

And finally, you'll **upload this trained agent to the Hugging Face Hub , a free, open platform where people can share ML models, datasets, and demos.**

It's essential **to master these elements** before diving into implementing Deep Reinforcement Learning agents. The goal of this chapter is to give you solid foundations.

After this unit, in a bonus unit, you'll be **able to train Huggy the Dog to fetch the stick and play with him** .



Figure 3: Huggy the Dog: still image from official course artwork (original video)

So let's get started!

2 What is Reinforcement Learning?

To understand Reinforcement Learning, let's start with the big picture.

2.1 The big picture

The idea behind Reinforcement Learning is that an agent (an AI) will learn from the environment by **interacting with it** (through trial and error) and **receiving rewards** (negative or positive) as feedback for performing actions.

Learning from interactions with the environment **comes from our natural experiences.**

For instance, imagine putting your little brother in front of a video game he never played, giving him a controller, and leaving him alone.

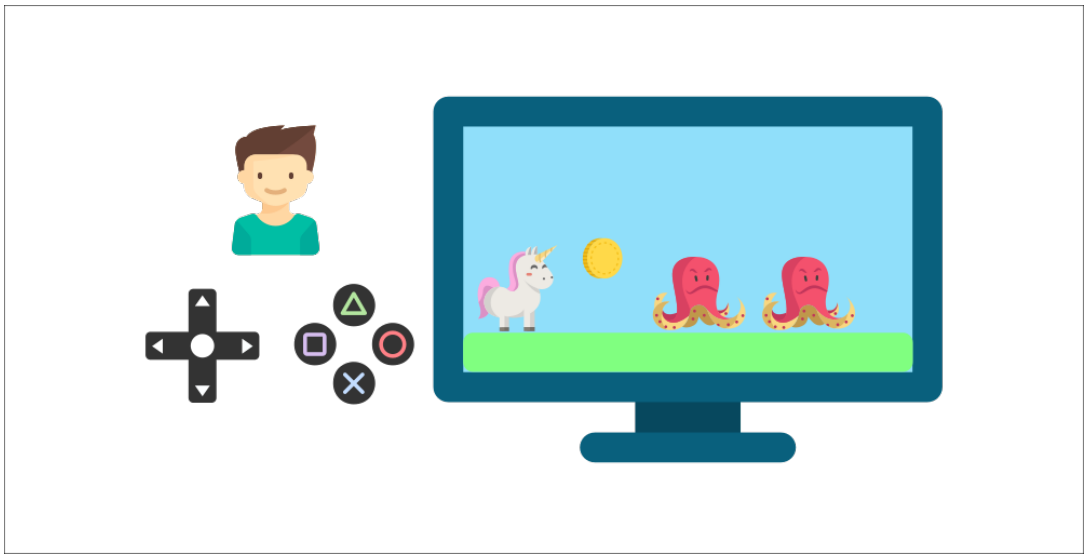


Figure 4: First action: the player moves right in the game

Your brother will interact with the environment (the video game) by pressing the right button (action). He got a coin, that's a +1 reward. It's positive, he just understood that in this game **he must get the coins.**

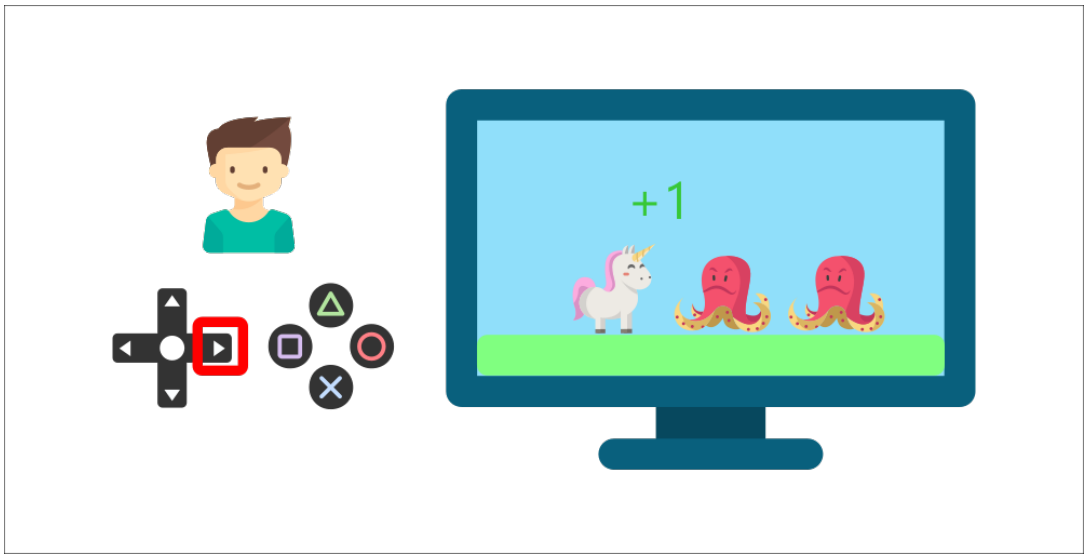


Figure 5: Positive reward: collecting a coin

But then, **he presses the right button again** and he touches an enemy. He just died, so that's a -1 reward.

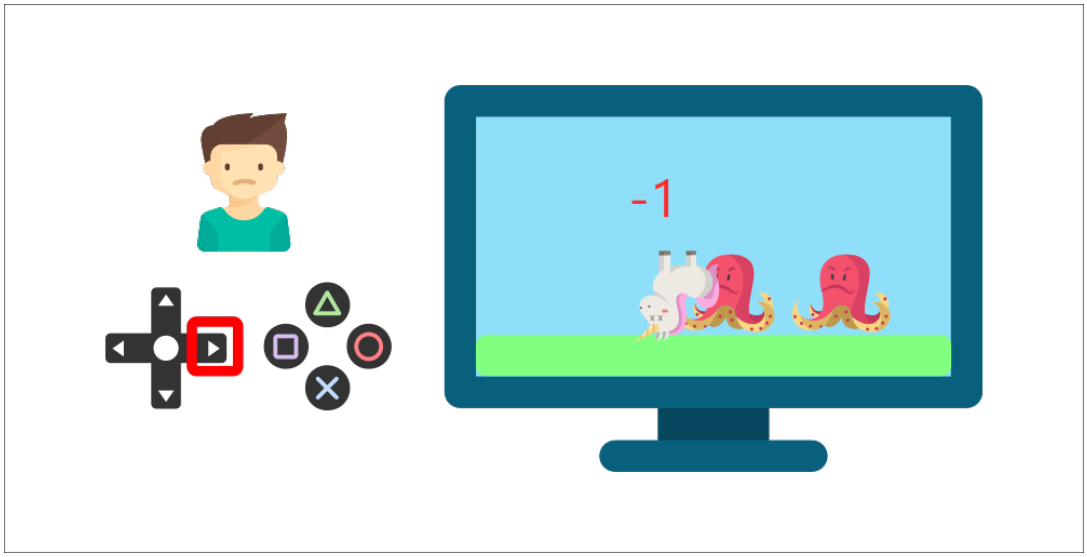


Figure 6: Negative reward: encountering an enemy

By interacting with his environment through trial and error, your little brother understands that **he needs to get coins in this environment but avoid the enemies.**

Without any supervision, the child will get better and better at playing the game.

That's how humans and animals learn, **through interaction.** Reinforcement Learning is just a **computational approach of learning from actions.**

A formal definition

We can now make a formal definition:

> **Course note.** Reinforcement learning is a framework for solving control tasks (also called decision problems) by building agents that learn from the environment by interacting with it through trial and error and receiving rewards (positive or negative) as unique feedback.

But how does Reinforcement Learning work?

3 The Reinforcement Learning Framework

3.1 The RL Process

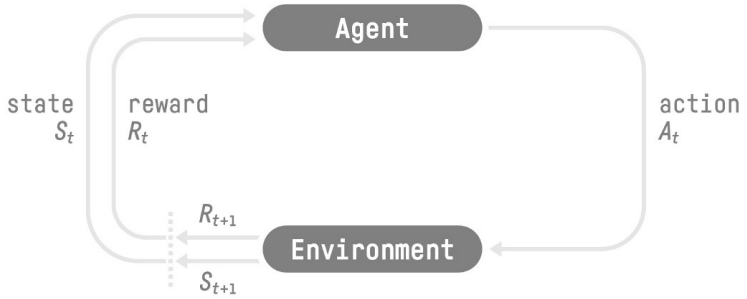


Figure 7: The RL Process: a loop of state, action, reward and next state Source: Reinforcement Learning: An Introduction, Richard Sutton and Andrew G. Barto

Source: http://incompleteideas.net/book/RLbook2020.pdf

To understand the RL process, let's imagine an agent learning to play a platform game:

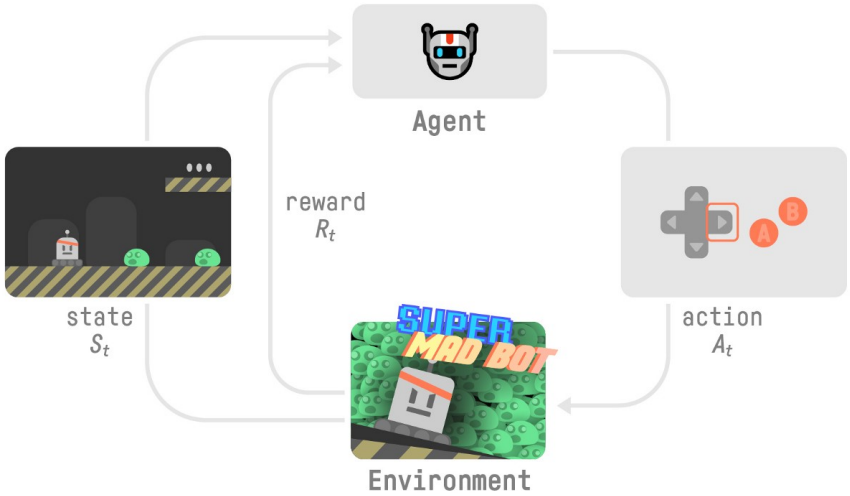


Figure 8: RL interaction cycle illustrated with a video-game environment

- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0**,** the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead *(Positive Reward +1)*.

This RL loop outputs a sequence of **state, action, reward and next state.**



Figure 9: Transition tuple: state, action, reward and next state

The agent's goal is to *maximize* its cumulative reward, **called the expected return.**

3.2 The reward hypothesis: the central idea of Reinforcement Learning

⇒ Why is the goal of the agent to maximize the expected return?

Because RL is based on the **reward hypothesis**, which is that all goals can be described as the **maximization of the expected return** (expected cumulative reward).

That's why in Reinforcement Learning, **to have the best behavior,** we aim to learn to take actions that **maximize the expected cumulative reward.**

3.3 Markov Property

In papers, you'll see that the RL process is called a **Markov Decision Process** (MDP).

We'll talk again about the Markov Property in the following units. But if you need to remember something today about it, it's this: the Markov Property implies that our agent needs **only the current state to decide** what action to take and **not the history of all the states and actions** they took before.

3.4 Observations/States Space

Observations/States are the **information our agent gets from the environment.** In the case of a video game, it can be a frame (a screenshot). In the case of the trading agent, it can be the value of a certain stock, etc.

There is a differentiation to make between *observation* and *state*, however:

- *State s*: is **a complete description of the state of the world** (there is no hidden information). In a fully observed environment.



Figure 10: In chess game, we receive a state from the environment since we have access to the whole check board information.

In a chess game, we have access to the whole board information, so we receive a state from the environment. In other words, the environment is fully observed.

- *Observation o*: is a **partial description of the state.** In a partially observed environment.



Figure 11: In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

In Super Mario Bros, we only see the part of the level close to the player, so we receive an observation.

In Super Mario Bros, we are in a partially observed environment. We receive an observation **since we only see a part of the level.**

Course note. In this course, we use the term "state" to denote both state and observation, but we will make the distinction in implementations.

To recap:

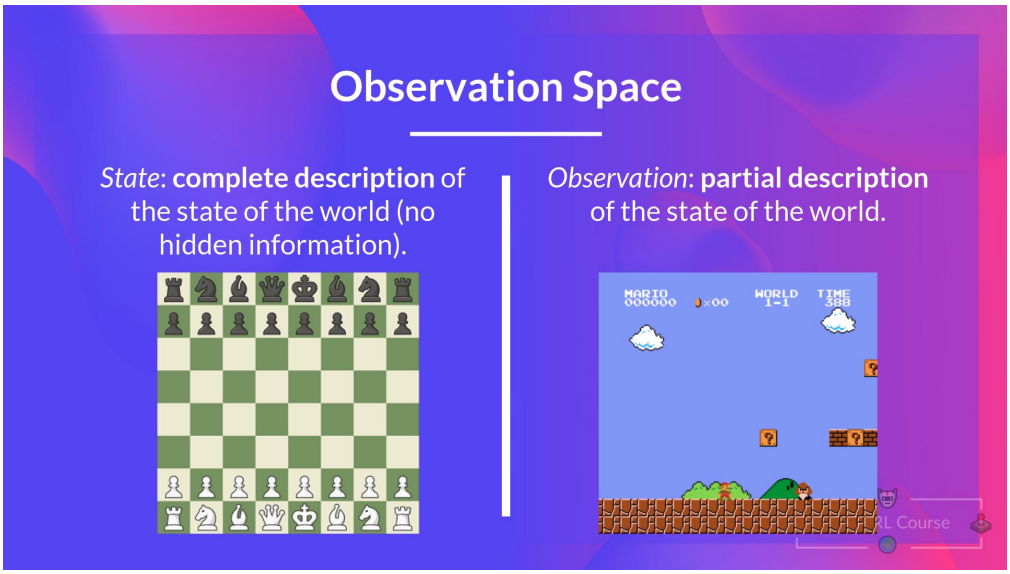


Figure 12: Complete state versus partial observation

3.5 Action Space

The Action space is the set of **all possible actions in an environment.**

The actions can come from a *discrete* or *continuous space*:

- *Discrete space*: the number of possible actions is **finite**.



Figure 13: In Super Mario Bros, we have only 4 possible actions: left, right, up (jumping) and down (crouching).

Again, in Super Mario Bros, we have a finite set of actions since we have only 4 directions.

- *Continuous space*: the number of possible actions is **infinite**.



Figure 14: A Self Driving Car agent has an infinite number of possible actions since it can turn left 20°, 21,1°, 21,2°, honk, turn right 20°...

To recap:



Figure 15: Discrete versus continuous action spaces

Taking this information into consideration is crucial because it will **have importance when choosing the RL algorithm in the future.**

3.6 Rewards and the discounting

The reward is fundamental in RL because it's **the only feedback** for the agent. Thanks to it, our agent knows **if the action taken was good or not.**

The cumulative reward at each time step **t** can be written as:

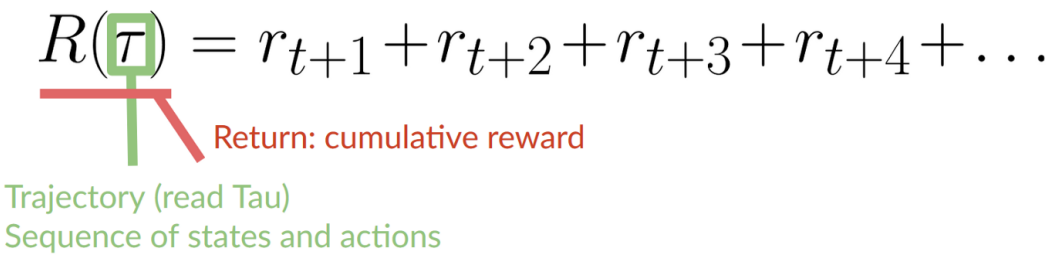


Figure 16: The cumulative reward equals the sum of all rewards in the sequence.

Which is equivalent to:

$$R(\tau) = \sum_{k=0}^{\infty} r_{t+k+1}$$

Figure 17: The cumulative reward = rt+1 (rt+k+1 = rt+0+1 = rt+1)+ rt+2 (rt+k+1 = rt+1+1 = rt+2) + ...

However, in reality, **we can't just add them like that.** The rewards that come sooner (at the beginning of the game) **are more likely to happen** since they are more predictable than the long-term future reward.

Let's say your agent is this tiny mouse that can move one tile each time step, and your opponent is the cat (that can move too). The mouse's goal is **to eat the maximum amount of cheese before being eaten by the cat.**

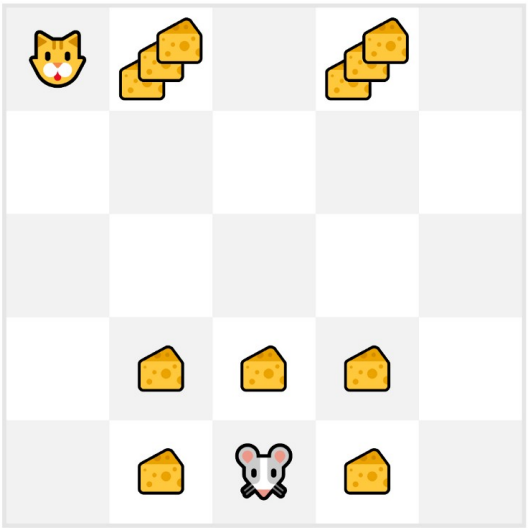


Figure 18: Cheese-and-cat illustration of delayed rewards

As we can see in the diagram, **it's more probable to eat the cheese near us than the cheese close to the cat** (the closer we are to the cat, the more dangerous it is).

Consequently, **the reward near the cat, even if it is bigger (more cheese), will be more discounted** since we're not really sure we'll be able to eat it.

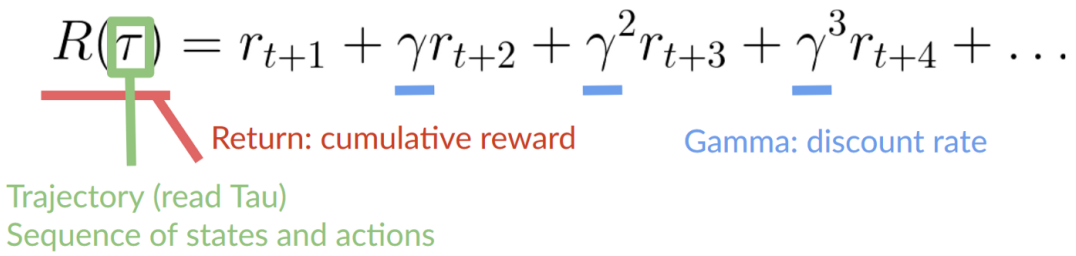
To discount the rewards, we proceed like this:

1. We define a discount rate called gamma. **It must be between 0 and 1.** Most of the time between **0.95 and 0.99**.

- The larger the gamma, the smaller the discount. This means our agent **cares more about the long-term reward.**
- On the other hand, the smaller the gamma, the bigger the discount. This means our **agent cares more about the short term reward (the nearest cheese).**

2. Then, each reward will be discounted by gamma to the exponent of the time step. As the time step increases, the cat gets closer to us, **so the future reward is less and less likely to happen.**

Our discounted expected cumulative reward is:



$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Figure 19: Discounted return with discount factor gamma

4 Type of tasks

A task is an **instance** of a Reinforcement Learning problem. We can have two types of tasks: **episodic** and **continuing**.

4.1 Episodic task

In this case, we have a starting point and an ending point **(a terminal state). This creates an episode**: a list of States, Actions, Rewards, and new States.

For instance, think about Super Mario Bros: an episode begins at the launch of a new Mario Level and ends **when you're killed or you reached the end of the level.**



Figure 20: Beginning of a new episode.

4.2 Continuing tasks

These are tasks that continue forever (**no terminal state**). In this case, the agent must **learn how to choose the best actions and simultaneously interact with the environment.**

For instance, an agent that does automated stock trading. For this task, there is no starting point and terminal state. **The agent keeps running until we decide to stop it.**



Figure 21: Stock trading as an example of a continuing task

To recap:

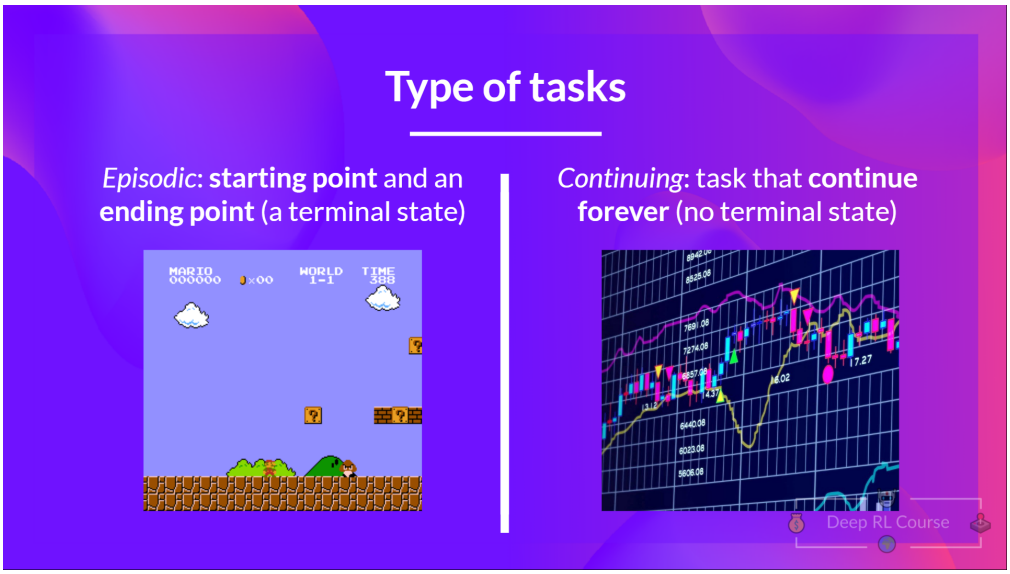


Figure 22: Episodic versus continuing tasks

5 The Exploration/Exploitation trade-off

Finally, before looking at the different methods to solve Reinforcement Learning problems, we must cover one more very important topic: *the exploration/exploitation trade-off.*

- *Exploration* is exploring the environment by trying random actions in order to **find more information about the environment.**
- *Exploitation* is **exploiting known information to maximize the reward.**

Remember, the goal of our RL agent is to maximize the expected cumulative reward. However, **we can fall into a common trap**.

Let's take an example:

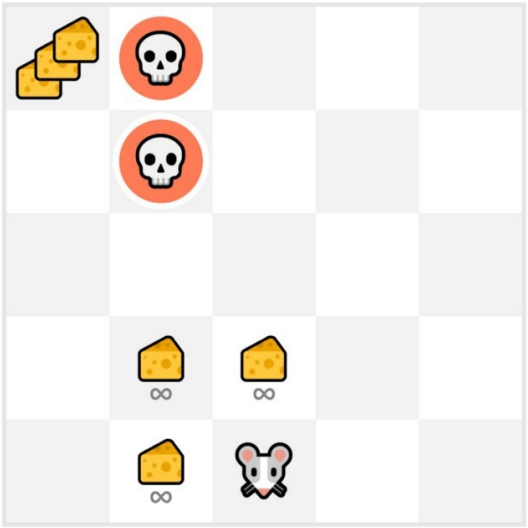


Figure 23: Cheese maze: local versus distant rewards in the exploration–exploitation trade-off

In this game, our mouse can have an **infinite amount of small cheese** (+1 each). But at the top of the maze, there is a gigantic sum of cheese (+1000).

However, if we only focus on exploitation, our agent will never reach the gigantic sum of cheese. Instead, it will only exploit **the nearest source of rewards,** even if this source is small (exploitation).

But if our agent does a little bit of exploration, it can **discover the big reward** (the pile of big cheese).

This is what we call the exploration/exploitation trade-off. We need to balance how much we **explore the environment** and how much we **exploit what we know about the environment.**

Therefore, we must **define a rule that helps to handle this trade-off**. We'll see the different ways to handle it in the future units.

If it's still confusing, **think of a real problem: the choice of picking a restaurant:**

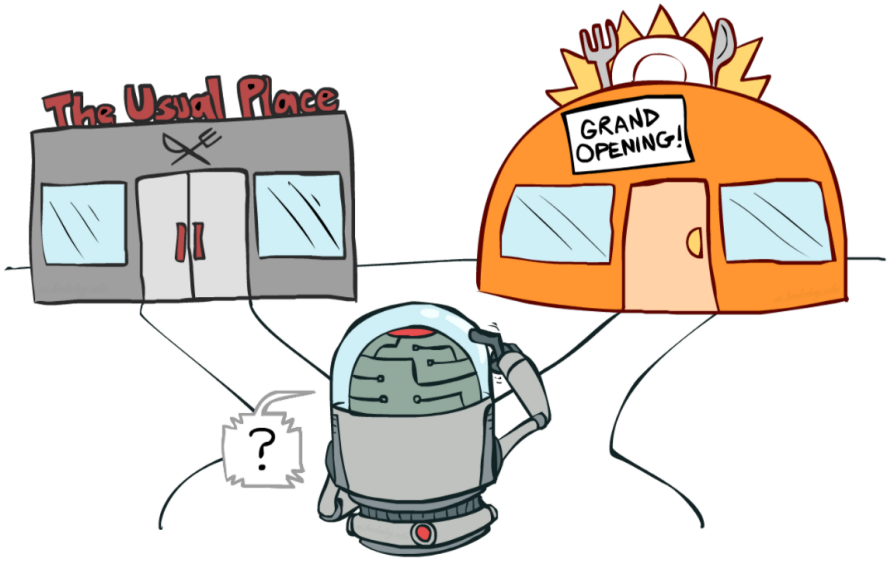


Figure 24: Exploration versus exploitation: familiar restaurant or unknown alternative — Source: Berkley AI Course

Source: https://inst.eecs.berkeley.edu/~cs188/sp20/assets/lecture/lec15_6up.pdf

- *Exploitation*: You go to the same one that you know is good every day and **take the risk to miss another better restaurant.**
- *Exploration*: Try restaurants you never went to before, with the risk of having a bad experience **but the probable opportunity of a fantastic experience.**

To recap:

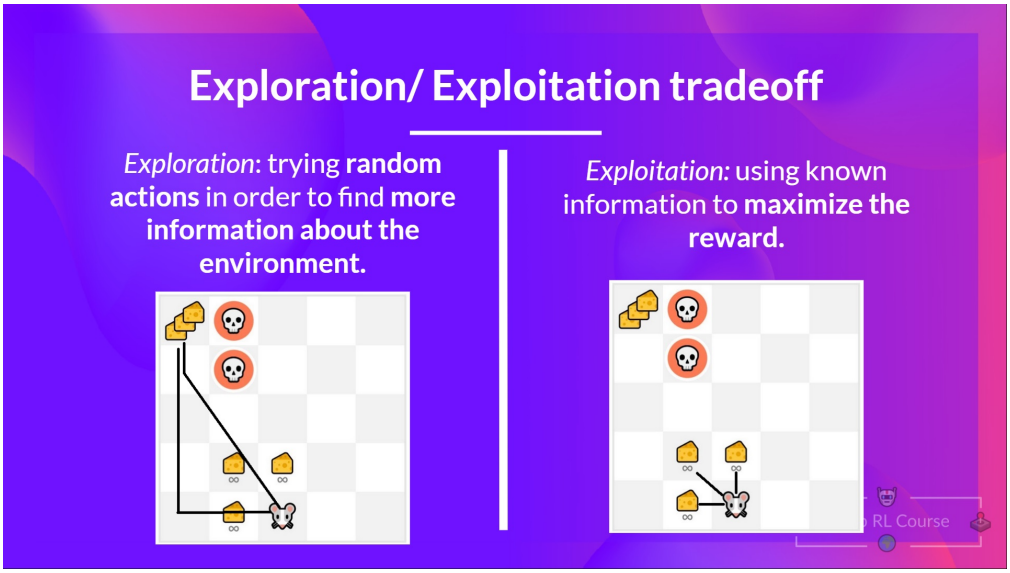


Figure 25: Exploration versus exploitation: summary diagram

6 Two main approaches for solving RL problems

Course note. Now that we learned the RL framework, how do we solve the RL problem?

In other words, how do we build an RL agent that can **select the actions that maximize its expected cumulative reward?**

6.1 The Policy π: the agent's brain

The Policy π is the **brain of our Agent**, it's the function that tells us what **action to take given the state we are in.** So it **defines the agent's behavior** at a given time.

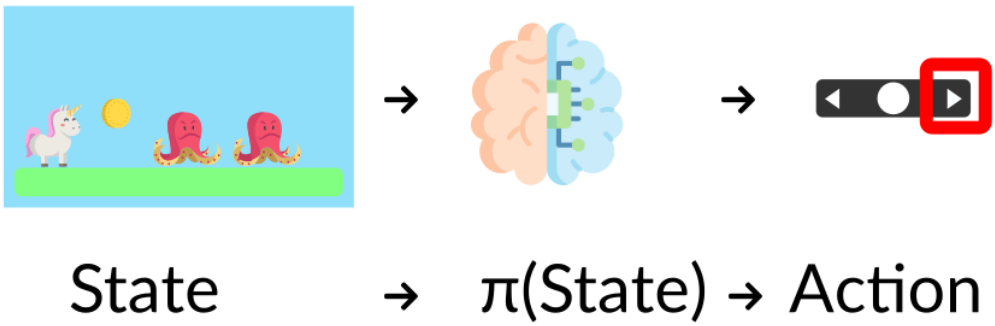


Figure 26: Think of policy as the brain of our agent, the function that will tell us the action to take given a state

This Policy **is the function we want to learn**, our goal is to find the optimal policy π^*, the policy that **maximizes expected return** when the agent acts according to it. We find this π^* **through training.**

There are two approaches to train our agent to find this optimal policy π^*:

- **Directly,** by teaching the agent to learn which **action to take,** given the current state: **Policy-Based Methods.**
- Indirectly, **teach the agent to learn which state is more valuable** and then take the action that **leads to the more valuable states**: Value-Based Methods.

6.2 Policy-Based Methods

In Policy-Based methods, **we learn a policy function directly.**

This function will define a mapping from each state to the best corresponding action. Alternatively, it could define **a probability distribution over the set of possible actions at that state.**

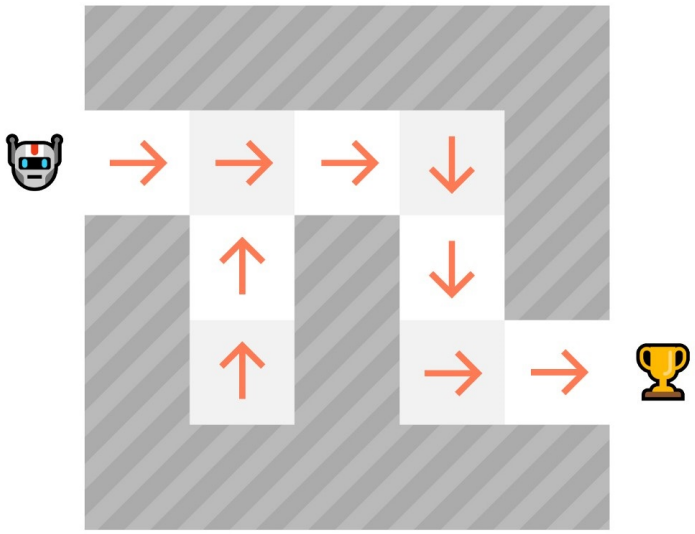


Figure 27: As we can see here, the policy (deterministic) directly indicates the action to take for each step.

We have two types of policies:

- *Deterministic*: a policy at a given state **will always return the same action.**

$$a = \pi(s)$$

Figure 28: action = policy(state)

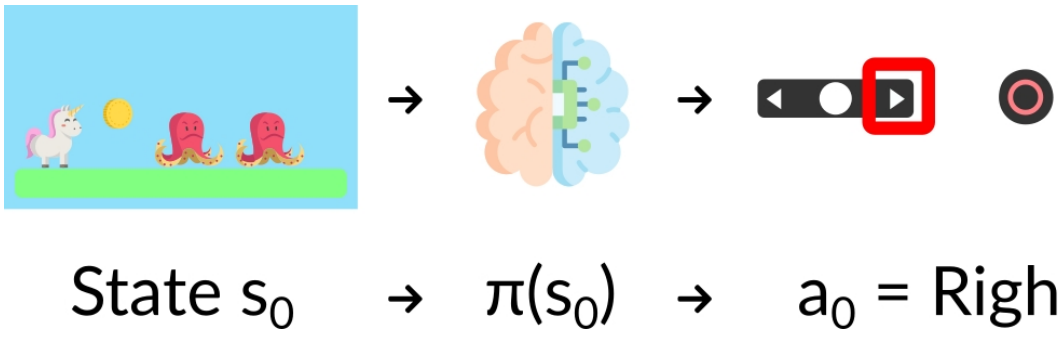


Figure 29: Stochastic policy: actions are sampled from a conditional probability distribution

- *Stochastic*: outputs **a probability distribution over actions.**

$$\pi(a|s) = \boxed{P[A|s]}$$

Probability Distribution over the set of actions given the state

Figure 30: policy(actions | state) = probability distribution over the set of actions given the current state

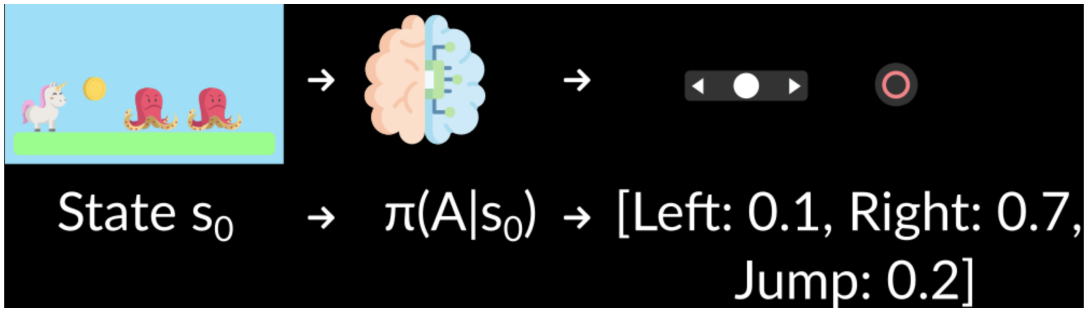


Figure 31: Given an initial state, our stochastic policy will output probability distributions over the possible actions at that state.

If we recap:

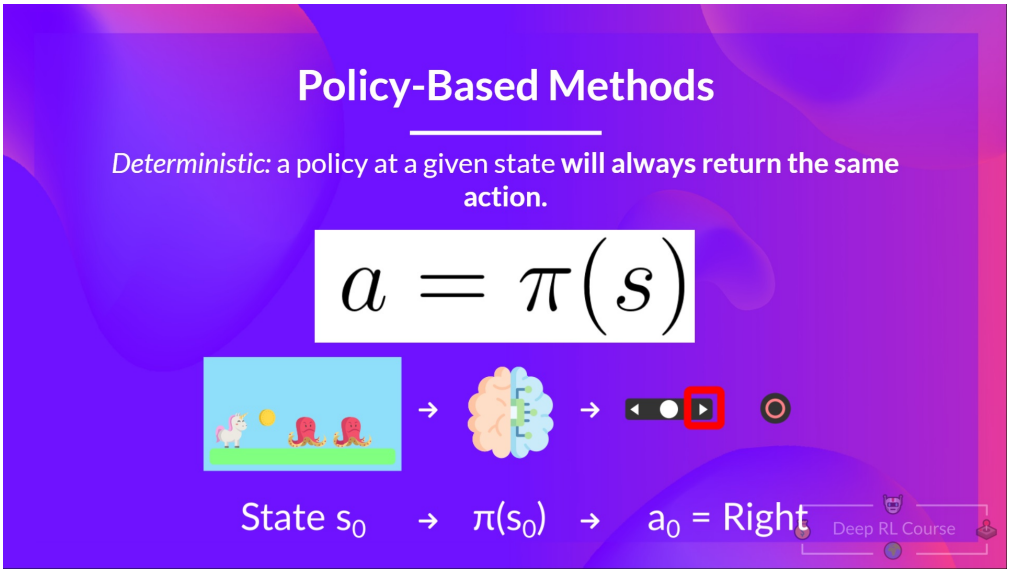


Figure 32: Policy-based methods: deterministic policies

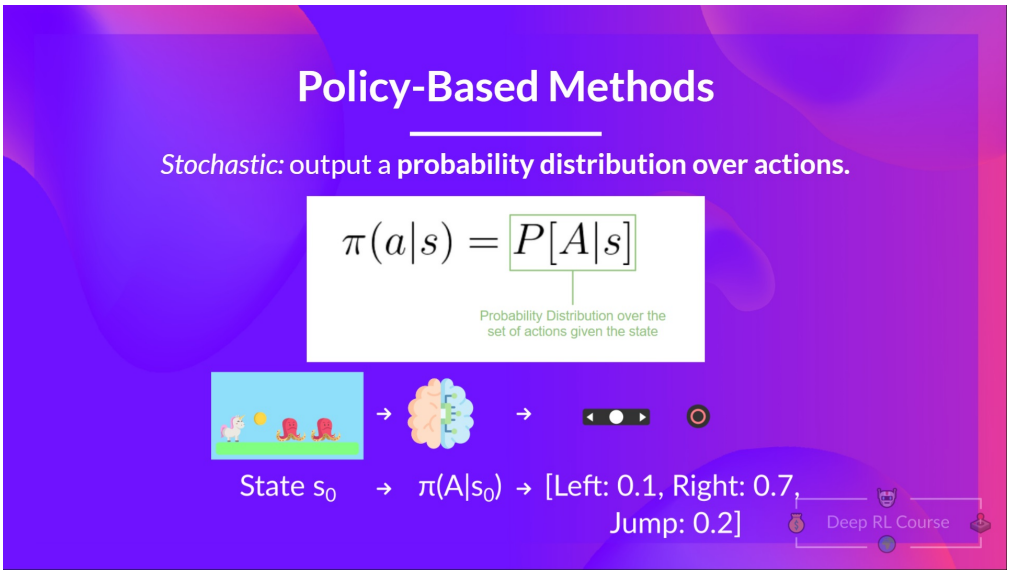


Figure 33: Policy-based methods: stochastic policies

6.3 Value-based methods

In value-based methods, instead of learning a policy function, we **learn a value function** that maps a state to the expected value **of being at that state.**

The value of a state is the **expected discounted return** the agent can get if it **starts in that state, and then acts according to our policy.**

"Act according to our policy" just means that our policy is **"going to the state with the highest value".**

$$v_\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \mid S_t = s\right]$$

Value function — Expected discounted return — Starting at state s

Figure 34: State-value function and expected discounted return

Here we see that our value function **defined values for each possible state.**

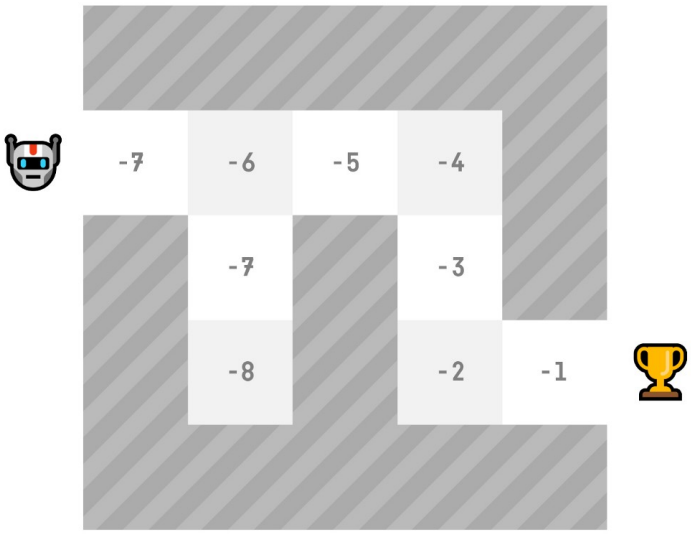


Figure 35: Thanks to our value function, at each step our policy will select the state with the biggest value defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

Thanks to our value function, at each step our policy will select the state with the biggest value defined by the value function: -7, then -6, then -5 (and so on) to attain the goal.

If we recap:

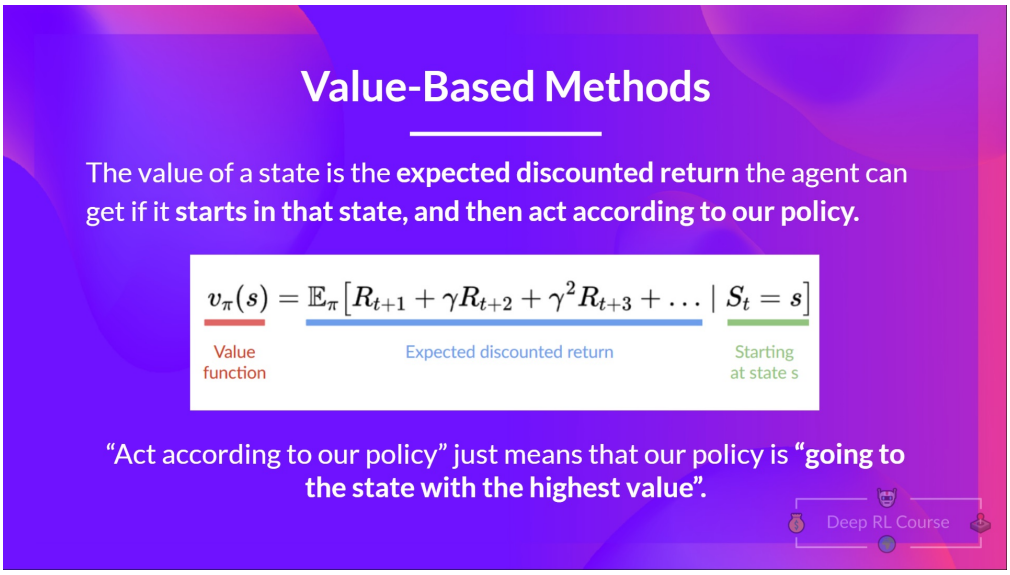


Figure 36: Value-based reinforcement learning: value-function overview

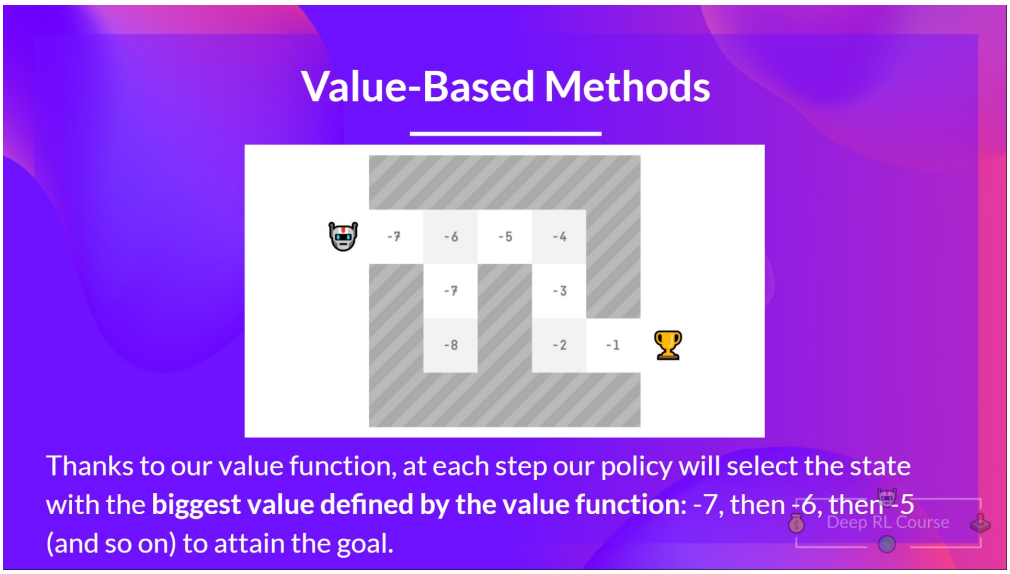


Figure 37: Value-based methods: selecting actions using state values

7 The "Deep" in Reinforcement Learning

Course note. What we've talked about so far is Reinforcement Learning. But where does the "Deep" come into play?

Deep Reinforcement Learning introduces **deep neural networks to solve Reinforcement Learning problems** — hence the name "deep".

For instance, in the next unit, we'll learn about two value-based algorithms: Q-Learning (classic Reinforcement Learning) and then Deep Q-Learning.

You'll see the difference is that, in the first approach, **we use a traditional algorithm** to create a Q table that helps us find what action to take for each state.

In the second approach, **we will use a Neural Network** (to approximate the Q value).

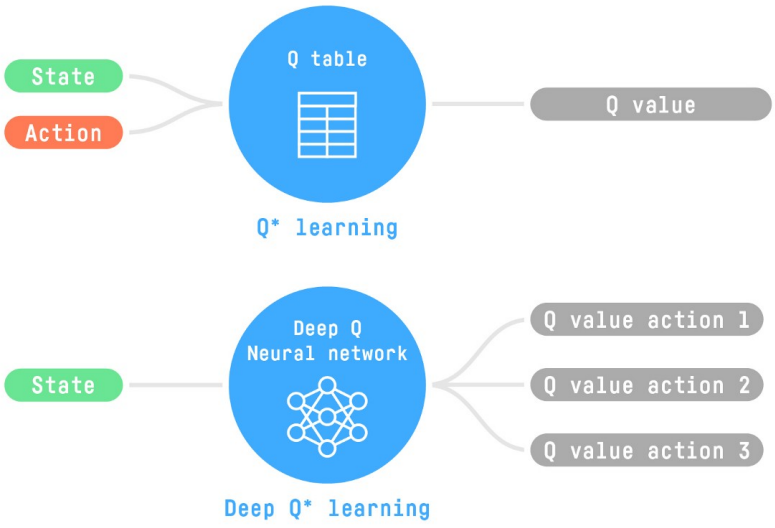


Figure 38: Schema inspired by the Q learning notebook by Udacity

If you are not familiar with Deep Learning you should definitely watch the FastAI Practical Deep Learning for Coders (Free).

8 Summary

That was a lot of information! Let's summarize:

- Reinforcement Learning is a computational approach of learning from actions. We build an agent that learns from the environment **by interacting with it through trial and error** and receiving rewards (negative or positive) as feedback.
- The goal of any RL agent is to maximize its expected cumulative reward (also called expected return) because RL is based on the **reward hypothesis**, which is that **all goals can be described as the maximization of the expected cumulative reward.**
- The RL process is a loop that outputs a sequence of **state, action, reward and next state.**
- To calculate the expected cumulative reward (expected return), we discount the rewards: the rewards that come sooner (at the beginning of the game) **are more probable to happen since they are more predictable than the long term future reward.**
- To solve an RL problem, you want to **find an optimal policy**. The policy is the "brain" of your agent, which will tell us **what action to take given a state.** The optimal policy is the one which **gives you the actions that maximize the expected return.**
- There are two ways to find your optimal policy:

  1. By training your policy directly: **policy-based methods.**
  2. By training a value function that tells us the expected return the agent will get at each state and use this function to define our policy: **value-based methods.**

- Finally, we speak about Deep RL because we introduce **deep neural networks to estimate the action to take (policy-based) or to estimate the value of a state (value-based)** hence the name "deep".

9 Glossary

This is a community-created glossary. Contributions are welcome!

Agent

An agent learns to **make decisions by trial and error, with rewards and punishments from the surroundings**.

Environment

An environment is a simulated world **where an agent can learn by interacting with it**.

Markov Property

It implies that the action taken by our agent is **conditional solely on the present state and independent of the past states and actions**.

Observations/State

- **State**: Complete description of the state of the world.
- **Observation**: Partial description of the state of the environment/world.

Actions

- **Discrete Actions**: Finite number of actions, such as left, right, up, and down.
- **Continuous Actions**: Infinite possibility of actions; for example, in the case of self-driving cars, the driving scenario has an infinite possibility of actions occurring.

Rewards and Discounting

- **Rewards**: Fundamental factor in RL. Tells the agent whether the action taken is good/bad.
- RL algorithms are focused on maximizing the **cumulative reward**.
- **Reward Hypothesis**: RL problems can be formulated as a maximisation of (cumulative) return.
- **Discounting** is performed because rewards obtained at the start are more likely to happen as they are more predictable than long-term rewards.

Tasks

- **Episodic**: Has a starting point and an ending point.
- **Continuous**: Has a starting point but no ending point.

Exploration v/s Exploitation Trade-Off

- **Exploration**: It's all about exploring the environment by trying random actions and receiving feedback/returns/rewards from the environment.
- **Exploitation**: It's about exploiting what we know about the environment to gain maximum rewards.
- **Exploration-Exploitation Trade-Off**: It balances how much we want to **explore** the environment and how much we want to **exploit** what we know about the environment.

Policy

- **Policy**: It is called the agent's brain. It tells us what action to take, given the state.
- **Optimal Policy**: Policy that **maximizes** the **expected return** when an agent acts according to it. It is learned through *training*.

Policy-based Methods:

- An approach to solving RL problems.
- In this method, the Policy is learned directly.
- Will map each state to the best corresponding action at that state. Or a probability distribution over the set of possible actions at that state.

Value-based Methods:

- Another approach to solving RL problems.
- Here, instead of training a policy, we train a **value function** that maps each state to the expected value of being in that state.

Contributions are welcome

If you want to improve the course, you can open a Pull Request.

This glossary was made possible thanks to:

- @lucifermorningstar1305
- @daspartho
- @misza222

10 Train your first Deep Reinforcement Learning Agent

Original notebook: https://colab.research.google.com/github/huggingface/deep-rl-class/blob/main/notebooks/unit1/unit1.ipynb

Now that you've studied the bases of Reinforcement Learning, you're ready to train your first agent and share it with the community through the Hub : A Lunar Lander agent that will learn to land correctly on the Moon

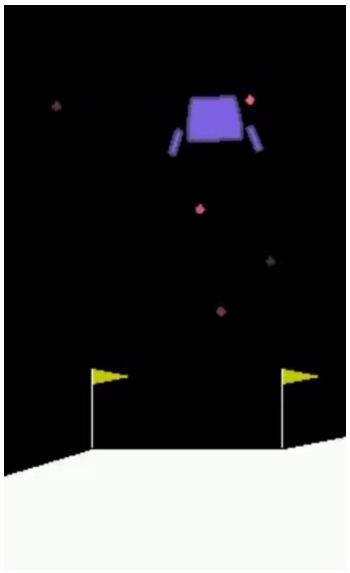


Figure 39: LunarLander environment: static scene from the official course thumbnail; the original animation is linked below (original GIF)

And finally, you'll **upload this trained agent to the Hugging Face Hub , a free, open platform where people can share ML models, datasets, and demos.**

Thanks to our leaderboard, you'll be able to compare your results with other classmates and exchange the best practices to improve your agent's scores. Who will win the challenge for Unit 1 ?

To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

If you don't find your model, go to the bottom of the page and click on the refresh button.

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

And you can check your progress here https : / / huggingface . co / spaces / ThomasSimonini / Check-my-progress-Deep-RL-Course

So let's get started!

To start the hands-on click on Open In Colab button :

Open original notebook in Google Colab

We strongly **recommend students use Google Colab for the hands-on exercises** instead of running them on their personal computers.

By using Google Colab, **you can focus on learning and experimenting without worrying about the technical aspects** of setting up your environments.

11 Unit 1: Train your first Deep Reinforcement Learning Agent

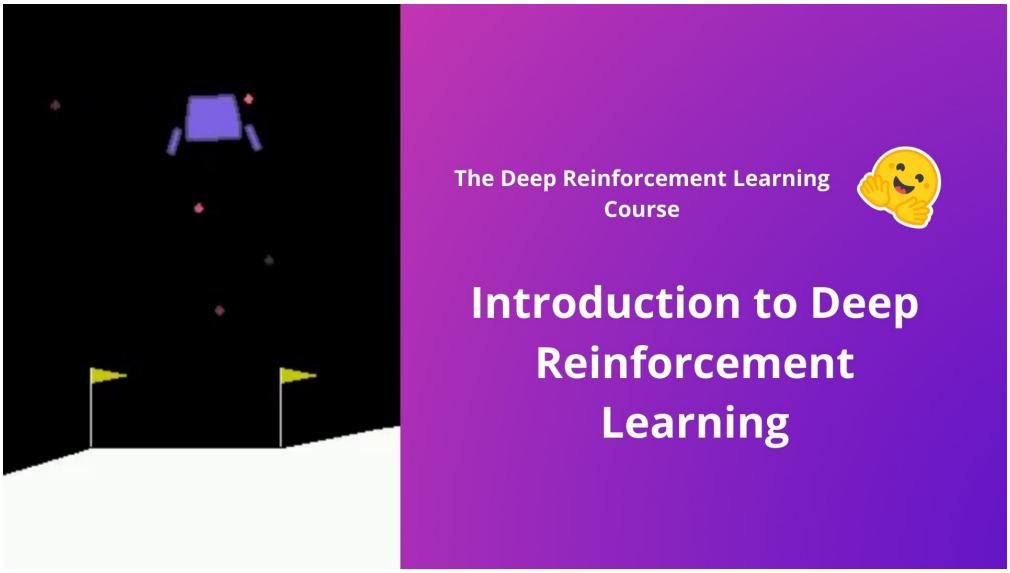


Figure 40: Unit 1: Introduction to Deep Reinforcement Learning (official course illustration)

In this notebook, you'll train your **first Deep Reinforcement Learning agent** a Lunar Lander agent that will learn to **land correctly on the Moon** . Using Stable-Baselines3 a Deep Reinforcement Learning library, share them with the community, and experiment with different configurations

The environment

- LunarLander-v2

The library used

- Stable-Baselines3

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the Github Repo.

11.1 Objectives of this notebook

At the end of the notebook, you will:

- Be able to use **Gymnasium**, the environment library.
- Be able to use **Stable-Baselines3**, the deep reinforcement learning library.
- Be able to **push your trained agent to the Hub** with a nice video replay and an evaluation score .

11.2 This notebook is from Deep Reinforcement Learning Course



Figure 41: Deep Reinforcement Learning Course overview

In this free course, you will:

- Study Deep Reinforcement Learning in **theory and practice**.
- Learn to **use famous Deep RL libraries** such as Stable Baselines3, RL Baselines3 Zoo, CleanRL and Sample Factory 2.0.
- Train **agents in unique environments**
- **Earn a certificate of completion** by completing 80% of the assignments.

And more!

Check the syllabus https://simoninithomas.github.io/deep-rl-course

Don't forget to **sign up to the course** (we are collecting your email to be able to **send you the links when each Unit is published and give you information about the challenges and updates).**

The best way to keep in touch and ask questions is **to join our discord server** to exchange with the community and with us https://discord.gg/ydHrjt3WP5

11.3 Prerequisites

Before diving into the notebook, you need to:

Read Unit 0 that gives you all the **information about the course and helps you to onboard**

Develop an understanding of the foundations of Reinforcement learning (MC, TD, Rewards hypothesis...) by reading Unit 1.

11.4 A small recap of Deep Reinforcement Learning

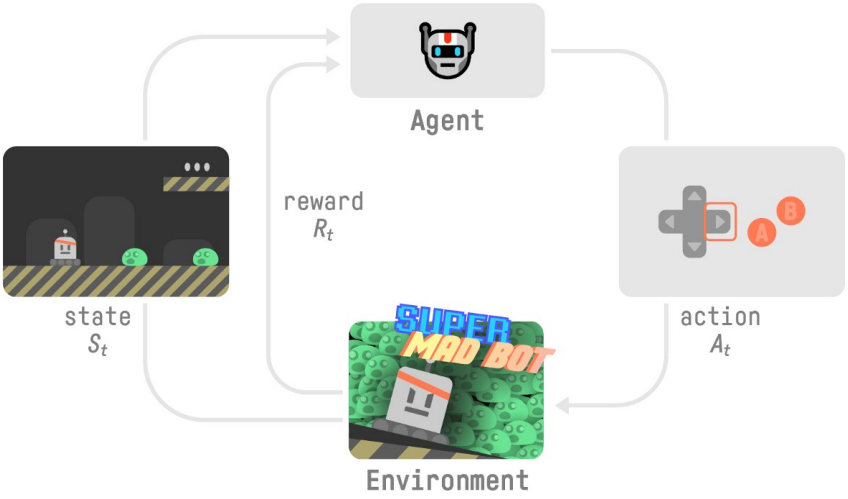


Figure 42: RL interaction cycle illustrated with a video-game environment

Let's do a small recap on what we learned in the first Unit:

- Reinforcement Learning is a **computational approach to learning from actions**. We build an agent that learns from the environment by **interacting with it through trial and error** and receiving rewards (negative or positive) as feedback.
- The goal of any RL agent is to **maximize its expected cumulative reward** (also called expected return) because RL is based on the *reward hypothesis*, which is that all goals can be described as the maximization of an expected cumulative reward.
- The RL process is a **loop that outputs a sequence of state, action, reward, and next state**.
- To calculate the expected cumulative reward (expected return), **we discount the rewards**: the rewards that come sooner (at the beginning of the game) are more probable to happen since they are more predictable than the long-term future reward.
- To solve an RL problem, you want to **find an optimal policy**; the policy is the "brain" of your AI that will tell us what action to take given a state. The optimal one is the one that gives you the actions that max the expected return.

There are **two** ways to find your optimal policy:

- By **training your policy directly**: policy-based methods.
- By **training a value function** that tells us the expected return the agent will get at each state and use this function to define our policy: value-based methods.
- Finally, we spoke about Deep RL because **we introduce deep neural networks to estimate the action to take (policy-based) or to estimate the value of a state (value-based) hence the name "deep."**

12 Let's train our first Deep Reinforcement Learning agent and upload it to the Hub

12.1 Get a certificate

To validate this hands-on for the certification process, you need to push your trained model to the Hub and **get a result of >= 200**.

To find your result, go to the leaderboard and find your model, **the result = mean_reward - std of reward**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

12.2 Set the GPU

- To **accelerate the agent's training, we'll use a GPU**. To do that, go to `Runtime > Change Runtime type`

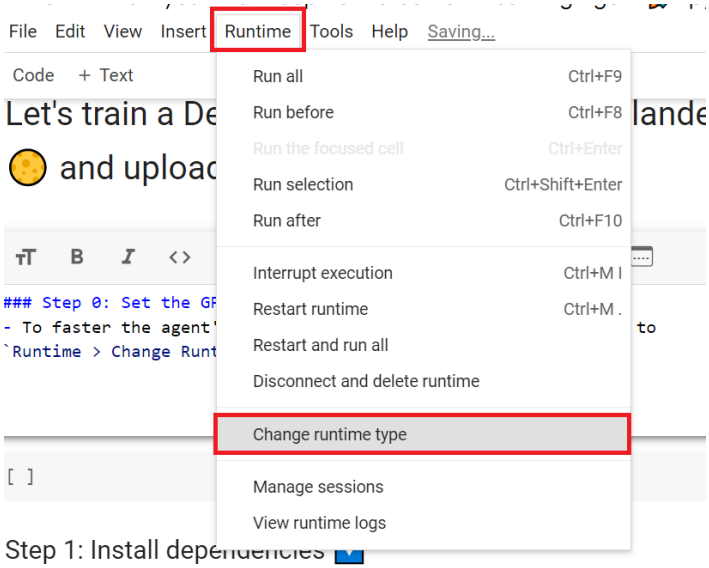


Figure 43: Google Colab menu for changing the runtime type

- `Hardware Accelerator > GPU`

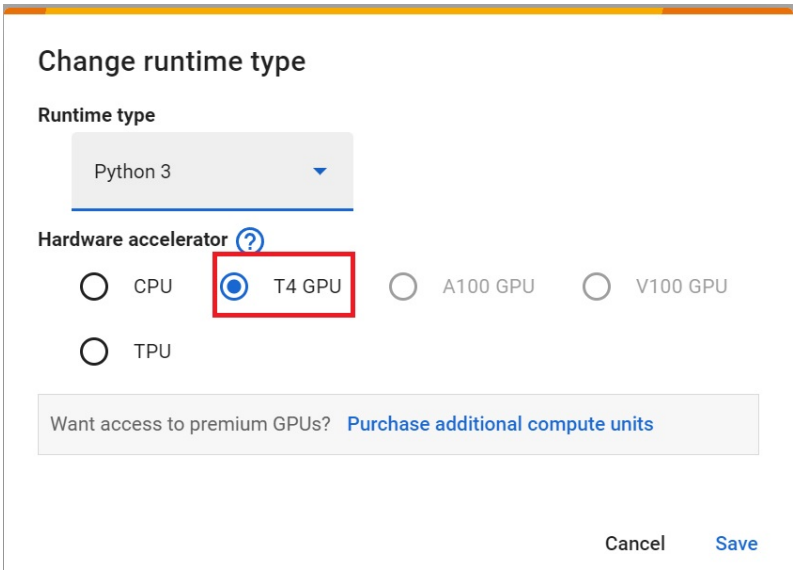


Figure 44: Google Colab GPU hardware accelerator selection

12.3 Install dependencies and create a virtual screen

The first step is to install the dependencies, we'll install multiple ones.

- `gymnasium[box2d]`: Contains the LunarLander-v2 environment
- `stable-baselines3[extra]`: The deep reinforcement learning library.
- `huggingface_sb3`: Additional code for Stable-baselines3 to load and upload models from the Hugging Face Hub.

To make things easier, we created a script to install all these dependencies.

```
apt install swig cmake
```

```
pip install -r https://raw.githubusercontent.com/huggingface/deep-rl-class/main/notebooks/unit1/
    requirements-unit1.txt
```

During the notebook, we'll need to generate a replay video. To do so, with colab, **we need to have a virtual screen to be able to render the environment** (and thus record the frames).

Hence the following cell will install virtual screen libraries and create and run a virtual screen

```
sudo apt-get update
apt install python3-opengl
apt install ffmpeg
apt install xvfb
pip3 install pyvirtualdisplay
```

To make sure the new installed libraries are used, **sometimes it's required to restart the notebook runtime**. The next cell will force the **runtime to crash, so you'll need to connect again and run the code starting from here**. Thanks to this trick, **we will be able to run our virtual screen.**

```
import os

os.kill(os.getpid(), 9)
```

```
# Virtual display
from pyvirtualdisplay import Display

virtual_display = Display(visible=0, size=(1400, 900))
virtual_display.start()
```

12.4 Import the packages

One additional library we import is huggingface_hub **to be able to upload and download trained models from the hub**.

The Hugging Face Hub works as a central place where anyone can share and explore models and datasets. It has versioning, metrics, visualizations and other features that will allow you to easily collaborate with others.

You can see here all the Deep reinforcement Learning models available here https://huggingface.co/models?pipeline_tag=reinforcement-learning&sort=downloads

```
import gymnasium

from huggingface_sb3 import load_from_hub, package_to_hub
from huggingface_hub import (
    notebook_login,
)  # To log to our Hugging Face account to be able to upload models to the Hub.
```

```
from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.monitor import Monitor
```

12.5 Understand Gymnasium and how it works

The library containing our environment is called Gymnasium. **You'll use Gymnasium a lot in Deep Reinforcement Learning.**

Gymnasium is the **new version of Gym library** maintained by the Farama Foundation.

The Gymnasium library provides two things:

- An interface that allows you to **create RL environments**.
- A **collection of environments** (gym-control, atari, box2D...).

Let's look at an example, but first let's recall the RL loop.

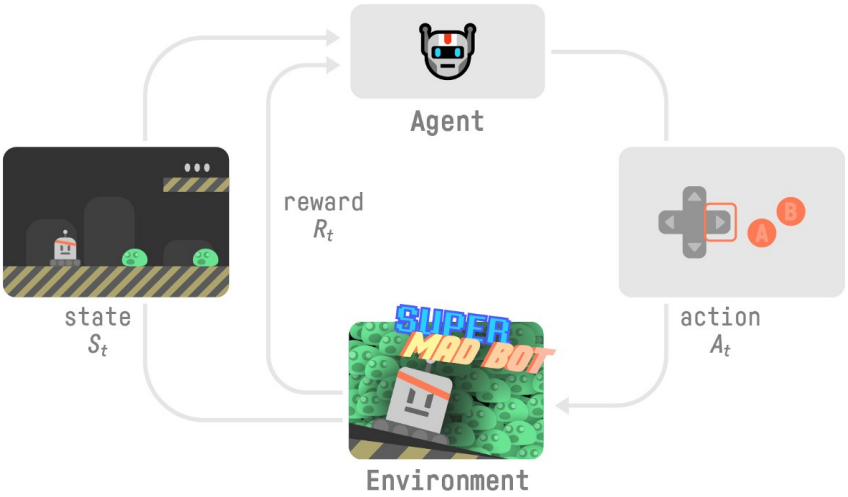


Figure 45: RL interaction cycle illustrated with a video-game environment

At each step:

- Our Agent receives a **state (S0)** from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state (S0),** the Agent takes an **action (A0)** — our Agent will move to the right.
- The environment transitions to a **new state (S1)** — new frame.
- The environment gives some **reward (R1)** to the Agent — we're not dead *(Positive Reward +1)*.

With Gymnasium:

1 We create our environment using `gymnasium.make()`

2 We reset the environment to its initial state with `observation = env.reset()`

At each step:

3 Get an action using our model (in our example we take a random action)

4 Using `env.step(action)`, we perform this action in the environment and get

- `observation`: The new state (st+1)
- `reward`: The reward we get after executing the action

- `terminated`: Indicates if the episode terminated (agent reach the terminal state)
- `truncated`: Introduced with this new version, it indicates a timelimit or if an agent go out of bounds of the environment for instance.
- `info`: A dictionary that provides additional information (depends on the environment).

For more explanations check this https://gymnasium.farama.org/api/env/#gymnasium.Env.step

If the episode is terminated:

- We reset the environment to its initial state with `observation = env.reset()`

Let's look at an example! Make sure to read the code

```
import gymnasium as gym

# First, we create our environment called LunarLander-v2
env = gym.make("LunarLander-v2")

# Then we reset this environment
observation, info = env.reset()

for _ in range(20):
    # Take a random action
    action = env.action_space.sample()
    print("Action taken:", action)

    # Do this action in the environment and get
    # next_state, reward, terminated, truncated and info
    observation, reward, terminated, truncated, info = env.step(action)

    # If the game is terminated (in our case we land, crashed) or truncated (timeout)
    if terminated or truncated:
        # Reset the environment
        print("Environment is reset")
        observation, info = env.reset()

env.close()
```

12.6 Create the LunarLander environment and understand how it works

The environment

In this first tutorial, we're going to train our agent, a Lunar Lander, **to land correctly on the moon**. To do that, the agent needs to learn **to adapt its speed and position (horizontal, vertical, and angular) to land correctly.**

A good habit when you start to use an environment is to check its documentation

https://gymnasium.farama.org/environments/box2d/lunar_lander/

Let's see what the Environment looks like:

```
# We create our environment with gym.make("<name_of_the_environment>")
env = gym.make("LunarLander-v2")
env.reset()
print("_____OBSERVATION SPACE_____ \n")
```

```
print("Observation Space Shape", env.observation_space.shape)
print("Sample observation", env.observation_space.sample())  # Get a random observation
```

We see with `Observation Space Shape (8,)` that the observation is a vector of size 8, where each value contains different information about the lander:

- Horizontal pad coordinate (x)
- Vertical pad coordinate (y)
- Horizontal speed (x)
- Vertical speed (y)
- Angle
- Angular speed
- If the left leg contact point has touched the land (boolean)
- If the right leg contact point has touched the land (boolean)

```
print("\n _____ACTION SPACE_____ \n")
print("Action Space Shape", env.action_space.n)
print("Action Space Sample", env.action_space.sample())  # Take a random action
```

The action space (the set of possible actions the agent can take) is discrete with 4 actions available :

- Action 0: Do nothing,
- Action 1: Fire left orientation engine,
- Action 2: Fire the main engine,
- Action 3: Fire right orientation engine.

Reward function (the function that will give a reward at each timestep) :

After every step a reward is granted. The total reward of an episode is the **sum of the rewards for all the steps within that episode**.

For each step, the reward:

- Is increased/decreased the closer/further the lander is to the landing pad.
- Is increased/decreased the slower/faster the lander is moving.
- Is decreased the more the lander is tilted (angle not horizontal).
- Is increased by 10 points for each leg that is in contact with the ground.
- Is decreased by 0.03 points each frame a side engine is firing.
- Is decreased by 0.3 points each frame the main engine is firing.

The episode receive an **additional reward of -100 or +100 points for crashing or landing safely respectively.**

An episode is **considered a solution if it scores at least 200 points.**

Vectorized Environment

- We create a vectorized environment (a method for stacking multiple independent environments into a single environment) of 16 environments, this way, **we'll have more diverse experiences during the training.**

```
# Create the environment
env = make_vec_env("LunarLander-v2", n_envs=16)
```

12.7 Create the Model

- We have studied our environment and we understood the problem: **being able to land the Lunar Lander to the Landing Pad correctly by controlling left, right and main orientation engine**. Now let's build the algorithm we're going to use to solve this Problem .
- To do so, we're going to use our first Deep RL library, Stable Baselines3 (SB3).
- SB3 is a set of **reliable implementations of reinforcement learning algorithms in PyTorch**.

A good habit when using a new library is to dive first on the documentation: https://stable-baselines3.readthedocs.io/en/master/ and then try some tutorials.



Figure 46: Stable-Baselines3 library illustration

To solve this problem, we're going to use SB3 **PPO**. PPO (aka Proximal Policy Optimization) is one of the SOTA (state of the art) Deep Reinforcement Learning algorithms that you'll study during this course.

PPO is a combination of:

- *Value-based reinforcement learning method*: learning an action-value function that will tell us the **most valuable action to take given a state and action**.
- *Policy-based reinforcement learning method*: learning a policy that will **give us a probability distribution over actions**.

Stable-Baselines3 is easy to set up:

1 You **create your environment** (in our case it was done above)

2 You define the **model you want to use and instantiate this model** `model = PPO("MlpPolicy")`

3 You **train the agent** with `model.learn` and define the number of training timesteps

```
# Create environment
env = gym.make('LunarLander-v2')

# Instantiate the agent
```

```
model = PPO('MlpPolicy', env, verbose=1)
# Train the agent
model.learn(total_timesteps=int(2e5))
```

```
# TODO: Define a PPO MlpPolicy architecture
# We use MultiLayerPerceptron (MLPPolicy) because the input is a vector,
# if we had frames as input we would use CnnPolicy
model =
```

Solution

```
# SOLUTION
# We added some parameters to accelerate the training
model = PPO(
    policy="MlpPolicy",
    env=env,
    n_steps=1024,
    batch_size=64,
    n_epochs=4,
    gamma=0.999,
    gae_lambda=0.98,
    ent_coef=0.01,
    verbose=1,
)
```

12.8 Train the PPO agent

- Let's train our agent for 1,000,000 timesteps, don't forget to use GPU on Colab. It will take approximately ~20min, but you can use fewer timesteps if you just want to try it out.
- During the training, take a break you deserved it

```
# TODO: Train it for 1,000,000 timesteps
# TODO: Specify file name for model and save the model to file
model_name = "ppo-LunarLander-v2"
```

Solution

```
# SOLUTION
# Train it for 1,000,000 timesteps
model.learn(total_timesteps=1000000)
# Save the model
model_name = "ppo-LunarLander-v2"
model.save(model_name)
```

12.9 Evaluate the agent

- Remember to wrap the environment in a Monitor.
- Now that our Lunar Lander agent is trained , we need to **check its performance**.
- Stable-Baselines3 provides a method to do that: `evaluate_policy`.
- To fill that part you need to check the documentation
- In the next step, we'll see **how to automatically evaluate and share your agent to compete in a leaderboard, but for now let's do it ourselves**

When you evaluate your agent, you should not use your training environment but create an evaluation environment.

```
# TODO: Evaluate the agent
# Create a new environment for evaluation
eval_env =

# Evaluate the model with 10 evaluation episodes and deterministic=True
mean_reward, std_reward =

# Print the results
```

Solution

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

- In my case, I got a mean reward of `200.20 +/- 20.80` after training for 1 million steps, which means that our lunar lander agent is ready to land on the moon .

12.10 Publish our trained model on the Hub

Now that we saw we got good results after the training, we can publish our trained model on the hub with one line of code.

The libraries documentation https://github.com/huggingface/huggingface_sb3/tree/main#hugging-face--x-stable-baselines3-v20

Here's an example of a Model Card (with Space Invaders):

By using `package_to_hub` **you evaluate, record a replay, generate a model card of your agent and push it to the hub**.

This way:

- You can **showcase our work**
- You can **visualize your agent playing**
- You can **share with the community an agent that others can use**
- You can **access a leaderboard to see how well your agent is performing compared to your classmates** https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

To be able to share your model with the community there are three more steps to follow:

1 (If it's not already done) create an account on Hugging Face https://huggingface.co/join

2 Sign in and then, you need to store your authentication token from the Hugging Face website.

- Create a new token (https://huggingface.co/settings/tokens) **with write role**

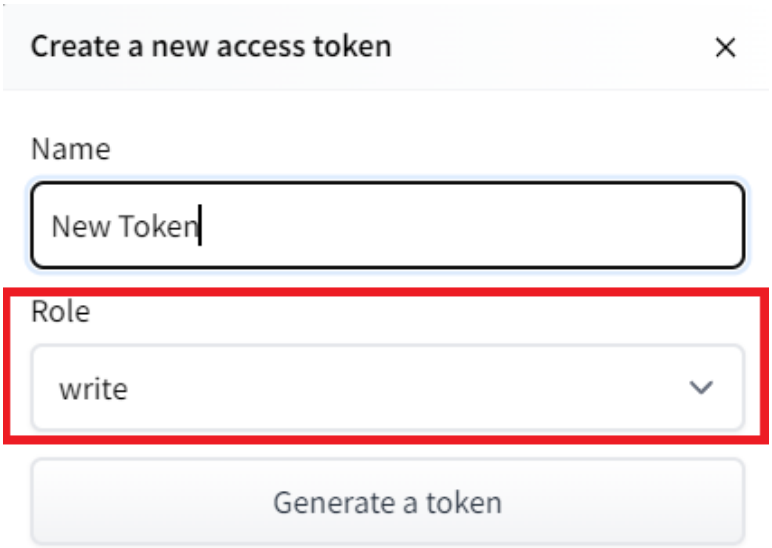


Figure 47: Create a Hugging Face Hub token with write permission

- Copy the token
- Run the cell below and paste the token

```
notebook_login()
!git config --global credential.helper store
```

If you don't want to use a Google Colab or a Jupyter Notebook, you need to use this command instead: `huggingface-cli login`

3 We're now ready to push our trained agent to the Hub using `package_to_hub()` function

Let's fill the `package_to_hub` function:

- `model`: our trained model.
- `model_name`: the name of the trained model that we defined in `model_save`
- `model_architecture`: the model architecture we used, in our case PPO
- `env_id`: the name of the environment, in our case `LunarLander-v2`
- `eval_env`: the evaluation environment defined in eval_env
- `repo_id`: the name of the Hugging Face Hub Repository that will be created/updated `(repo_id = {username}/{repo_name})`

A good name is `{username}/{model_architecture}-{env_id}`

- `commit_message`: message of the commit

```
import gymnasium as gym
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

## TODO: Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organization}/{
    repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
repo_id =

# TODO: Define the name of the environment
env_id =

# Create the evaluation env and set the render_mode="rgb_array"
```

```
eval_env = DummyVecEnv([lambda: gym.make(env_id, render_mode="rgb_array")])


# TODO: Define the model architecture we used
model_architecture = ""

## TODO: Define the commit message
commit_message = ""

# method save, evaluate, generate a model card and record a replay video of your agent before
    pushing the repo to the hub
package_to_hub(model=model, # Our trained model
               model_name=model_name, # The name of our trained model
               model_architecture=model_architecture, # The model architecture we used: in our case
     PPO
               env_id=env_id, # Name of the environment
               eval_env=eval_env, # Evaluation Environment
               repo_id=repo_id, # id of the model repository from the Hugging Face Hub (repo_id = {
    organization}/{repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
               commit_message=commit_message)
```

Solution

```
import gymnasium as gym

from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.env_util import make_vec_env

from huggingface_sb3 import package_to_hub

# PLACE the variables you've just defined two cells above
# Define the name of the environment
env_id = "LunarLander-v2"

# TODO: Define the model architecture we used
model_architecture = "PPO"

## Define a repo_id
## repo_id is the id of the model repository from the Hugging Face Hub (repo_id = {organization}/{
    repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
## CHANGE WITH YOUR REPO ID
repo_id = "ThomasSimonini/ppo-LunarLander-v2"  # Change with your repo id, you can't push with mine
     :)

## Define the commit message
commit_message = "Upload PPO LunarLander-v2 trained agent"

# Create the evaluation env and set the render_mode="rgb_array"
eval_env = DummyVecEnv([lambda: Monitor(gym.make(env_id, render_mode="rgb_array"))])

# PLACE the package_to_hub function you've just filled here
package_to_hub(
    model=model,  # Our trained model
    model_name=model_name,  # The name of our trained model
    model_architecture=model_architecture,  # The model architecture we used: in our case PPO
    env_id=env_id,  # Name of the environment
    eval_env=eval_env,  # Evaluation Environment
    repo_id=repo_id,  # id of the model repository from the Hugging Face Hub (repo_id = {
    organization}/{repo_name} for instance ThomasSimonini/ppo-LunarLander-v2
    commit_message=commit_message,
)
```

Congrats you've just trained and uploaded your first Deep Reinforcement Learning agent. The script above should have displayed a link to a model repository such as https://huggingface.co/osanseviero/test_sb3. When you go to this link, you can:

- See a video preview of your agent at the right.
- Click "Files and versions" to see all the files in the repository.
- Click "Use in stable-baselines3" to get a code snippet that shows how to load the model.
- A model card (`README.md` file) which gives a description of the model

Under the hood, the Hub uses git-based repositories (don't worry if you don't know what git is), which means you can update the model with new versions as you experiment and improve your agent.

Compare the results of your LunarLander-v2 with your classmates using the leaderboard https://huggingface.co/spaces/huggingface-projects/Deep-Reinforcement-Learning-Leaderboard

12.11 Load a saved LunarLander model from the Hub

Thanks to ironbar for the contribution.

Loading a saved model from the Hub is really easy.

You go to https://huggingface.co/models?library=stable-baselines3 to see the list of all the Stable-baselines3 saved models.

1. You select one and copy its repo_id

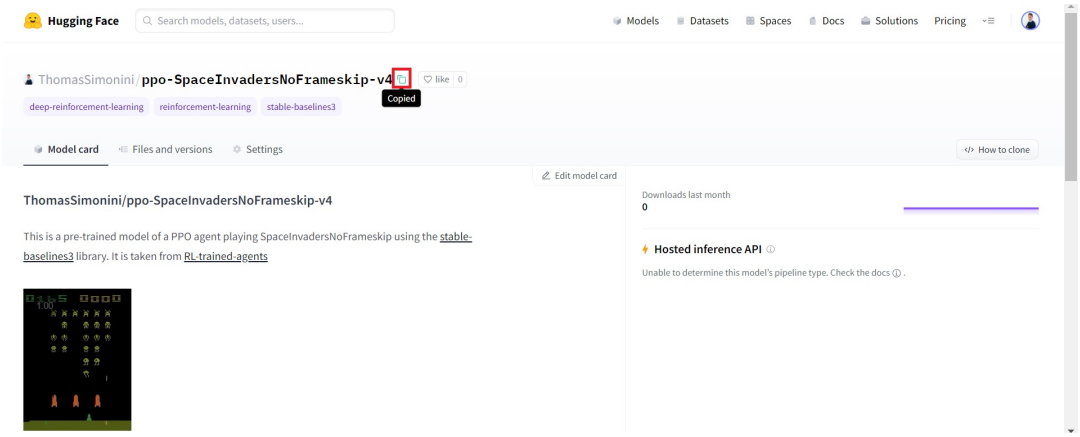


Figure 48: Copy the model repository identifier from Hugging Face Hub

2. Then we just need to use load_from_hub with:

- The repo_id
- The filename: the saved model inside the repo and its extension (*.zip)

Because the model I download from the Hub was trained with Gym (the former version of Gymnasium) we need to install shimmy a API conversion tool that will help us to run the environment correctly.

Shimmy Documentation: https://github.com/Farama-Foundation/Shimmy

```
!pip install shimmy
```

```
from huggingface_sb3 import load_from_hub

repo_id = "Classroom-workshop/assignment2-omar"  # The repo_id
filename = "ppo-LunarLander-v2.zip"  # The model filename.zip
```

```
# When the model was trained on Python 3.8 the pickle protocol is 5
# But Python 3.6, 3.7 use protocol 4
# In order to get compatibility we need to:
# 1. Install pickle5 (we done it at the beginning of the colab)
# 2. Create a custom empty object we pass as parameter to PPO.load()
custom_objects = {
    "learning_rate": 0.0,
    "lr_schedule": lambda _: 0.0,
    "clip_range": lambda _: 0.0,
}

checkpoint = load_from_hub(repo_id, filename)
model = PPO.load(checkpoint, custom_objects=custom_objects, print_system_info=True)
```

Let's evaluate this agent:

```
# @title
eval_env = Monitor(gym.make("LunarLander-v2"))
mean_reward, std_reward = evaluate_policy(model, eval_env, n_eval_episodes=10, deterministic=True)
print(f"mean_reward={mean_reward:.2f} +/- {std_reward}")
```

12.12 Some additional challenges

The best way to learn **is to try things by your own**! As you saw, the current agent is not doing great. As a first suggestion, you can train for more steps. With 1,000,000 steps, we saw some great results!

In the Leaderboard you will find your agents. Can you get to the top?

Here are some ideas to achieve so:

- Train more steps
- Try different hyperparameters for `PPO`. You can see them at https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html#parameters.
- Check the Stable-Baselines3 documentation and try another model such as DQN.
- **Push your new trained model** on the Hub

Compare the results of your LunarLander-v2 with your classmates using the leaderboard

Is moon landing too boring for you? Try to **change the environment**, why not use MountainCar-v0, CartPole-v1 or CarRacing-v0? Check how they work using the gym documentation and have fun .

Congrats on finishing this chapter! That was the biggest one, **and there was a lot of information.**

If you're still feel confused with all these elements...it's totally normal! **This was the same for me and for all people who studied RL.**

Take time to really **grasp the material before continuing and try the additional challenges**. It's important to master these elements and have a solid foundations.

Naturally, during the course, we're going to dive deeper into these concepts but **it's better to have a good understanding of them now before diving into the next chapters.**

Next time, in the bonus unit 1, you'll train Huggy the Dog to fetch the stick.



Figure 49: Huggy in the virtual training environment

12.13 Keep learning, stay awesome

13 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

Q1: What is Reinforcement Learning?

Q2: Define the RL Loop

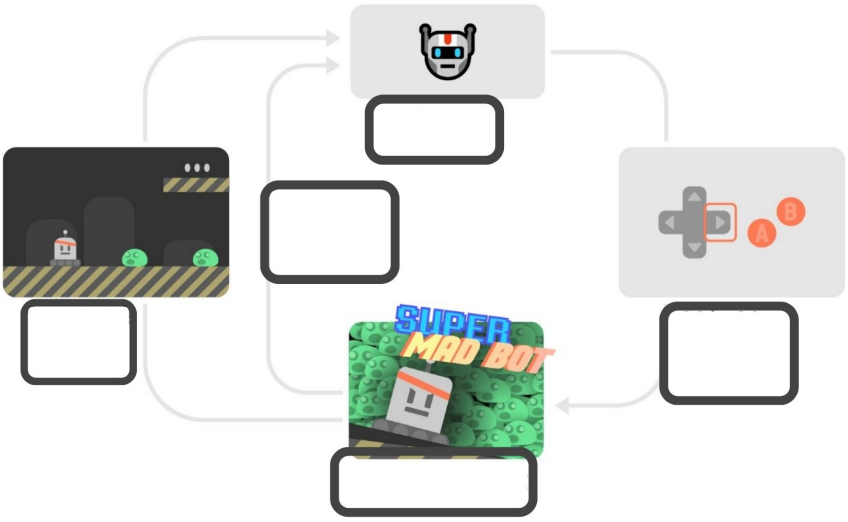


Figure 50: Quiz exercise: fill in the state, action, next state and reward of the RL loop

At every step:

- Our Agent receives _____ from the environment
- Based on that _____ the Agent takes an _____
- Our Agent will move to the right
- The Environment goes to a _____
- The Environment gives a _____ to the Agent

Select all correct choices (multiple answers may apply).

- **A.** an action a0, action a0, state s0, state s1, reward r1
- **B.** state s0, state s0, action a0, new state s1, reward r1
- **C.** a state s0, state s0, action a0, state s1, action a1

Q3: What's the difference between a state and an observation?

Select all correct choices (multiple answers may apply).

- **A.** The state is a complete description of the state of the world (there is no hidden information)
- **B.** The state is a partial description of the state
- **C.** The observation is a complete description of the state of the world (there is no hidden information)
- **D.** The observation is a partial description of the state
- **E.** We receive a state when we play with chess environment
- **F.** We receive an observation when we play with chess environment
- **G.** We receive a state when we play with Super Mario Bros
- **H.** We receive an observation when we play with Super Mario Bros

Q4: A task is an instance of a Reinforcement Learning problem. What are the two types of tasks?

Select all correct choices (multiple answers may apply).

- **A.** Episodic
- **B.** Recursive
- **C.** Adversarial
- **D.** Continuing

Q5: What is the exploration/exploitation tradeoff?

Q6: What is a policy?

Q7: What are value-based methods?

Q8: What are policy-based methods?

13.1 Answer Key and Explanations

Q1: What is Reinforcement Learning?

Reinforcement learning is a **framework for solving control tasks (also called decision problems)** by building agents that learn from the environment by interacting with it through trial and error and **receiving rewards (positive or negative) as unique feedback**.

Q2: Define the RL Loop

Answer: B

- **Choice A:** At every step: Our Agent receives **state s0** from the environment. Based on that **state s0** the Agent takes an **action a0**. Our Agent will move to the right. The Environment goes to a **new state s1**. The Environment gives **a reward r1** to the Agent.
- **Choice C:** At every step: Our Agent receives **state s0** from the environment. Based on that **state s0** the Agent takes an **action a0**. Our Agent will move to the right. The Environment goes to a **new state s1**. The Environment gives **a reward r1** to the Agent.

Q3: What's the difference between a state and an observation?

Answer: A, D, E, H

- **Choice E (correct):** Since we have access to the whole checkboard information.
- **Choice F:** Since we have access to the whole checkboard information.
- **Choice G:** We only see a part of the level close to the player, so we receive an observation.
- **Choice H (correct):** We only see a part of the level close to the player.

Q4: A task is an instance of a Reinforcement Learning problem. What are the two types of tasks?

Answer: A, D

- **Choice A (correct):** In Episodic task, we have a starting point and an ending point (a terminal state). This creates an episode: a list of States, Actions, Rewards, and new States. For instance, think about Super Mario Bros: an episode begin at the launch of a new Mario Level and ending when you're killed or you reached the end of the level.
- **Choice D (correct):** Continuing tasks are tasks that continue forever (no terminal state). In this case, the agent must learn how to choose the best actions and simultaneously interact with the environment.

Q5: What is the exploration/exploitation tradeoff?

In Reinforcement Learning, we need to **balance how much we explore the environment and how much we exploit what we know about the environment**.

- *Exploration* is exploring the environment by **trying random actions in order to find more information about the environment**.
- *Exploitation* is **exploiting known information to maximize the reward**.

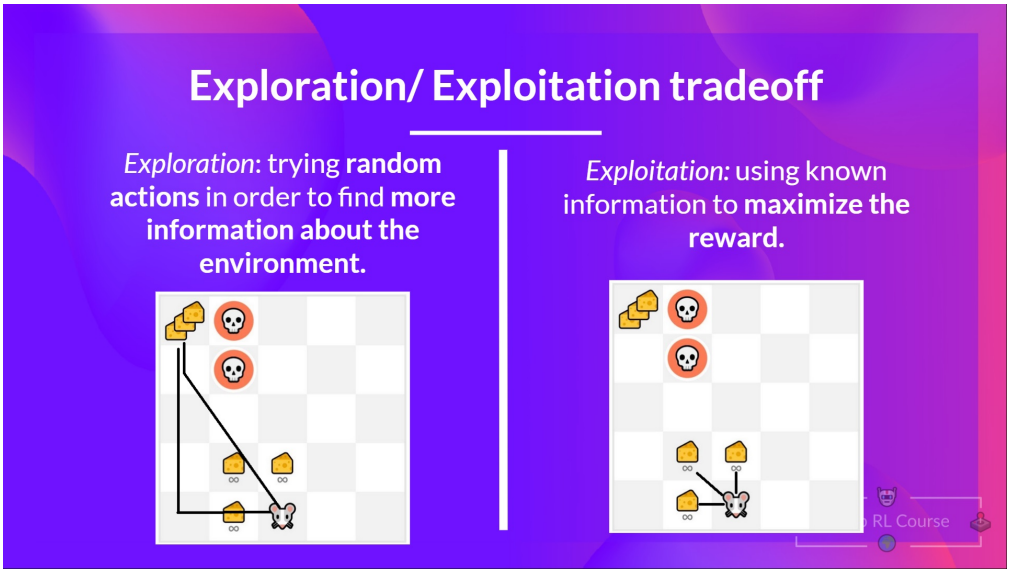


Figure 51: Exploration versus exploitation: summary diagram

Q6: What is a policy?

- The Policy π **is the brain of our Agent**. It's the function that tells us what action to take given the state we are in. So it defines the agent's behavior at a given time.

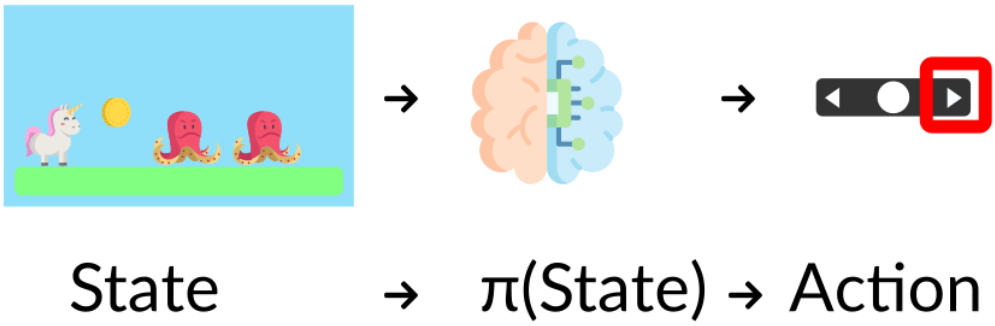


Figure 52: Policy maps the current state to an action

Q7: What are value-based methods?

- Value-based methods is one of the main approaches for solving RL problems.
- In Value-based methods, instead of training a policy function, **we train a value function that maps a state to the expected value of being at that state**.

Q8: What are policy-based methods?

- In *Policy-Based Methods*, we learn a **policy function directly**.
- This policy function will **map from each state to the best corresponding action at that state**. Or a **probability distribution over the set of possible actions at that state**.

Congrats on finishing this Quiz , if you missed some elements, take time to read again the chapter to reinforce () your knowledge, but **do not worry**: during the course we'll go over again of these concepts, and you'll **reinforce your theoretical knowledge with hands-on**.

14 Conclusion

Congrats on finishing this unit! **That was the biggest one**, and there was a lot of information. And congrats on finishing the tutorial. You've just trained your first Deep RL agents and shared them with the community!

It's **normal if you still feel confused by some of these elements**. This was the same for me and for all people who studied RL.

Take time to really grasp the material before continuing. It's important to master these elements and have a solid foundation before entering the fun part.

Naturally, during the course, we're going to use and explain these terms again, but it's better to understand them before diving into the next units.

In the next (bonus) unit, we're going to reinforce what we just learned by **training Huggy the Dog to fetch a stick**.

You will then be able to play with him .



Figure 53: Huggy the Dog: training environment shown in the bonus unit

Finally, we would love **to hear what you think of the course and how we can improve it**. If you have some feedback then, please fill this form

Keep Learning, stay awesome

15 Additional Readings

These are **optional readings** if you want to go deeper.

15.1 Deep Reinforcement Learning

- Reinforcement Learning: An Introduction, Richard Sutton and Andrew G. Barto Chapter 1, 2 and 3
- Foundations of Deep RL Series, L1 MDPs, Exact Solution Methods, Max-ent RL by Pieter Abbeel
- Spinning Up RL by OpenAI Part 1: Key concepts of RL

15.2 Gym

- Getting Started With OpenAI Gym: The Basic Building Blocks
- Make your own Gym custom environment